

Interview Script: Problems & Drafts Module

1. Introduction (The Big Picture)

"In ThinkStack, the core feature is the **5-Step Thinking Process**. Instead of just giving an answer, users go through: **Understanding, Breakdown, Approach, Solution, and Reflection**. My goal was to make this flow seamless, so users can stop anytime and resume exactly where they left off. I built this using the MERN stack."

2. Database Layer (The Foundation)

"I designed two main schemas in MongoDB to handle this:

- **Problem Model:** This is the static part. It stores the title, description, and the three hints that help the user.
 - **Draft Model:** This is the dynamic part. It tracks the user's progress. It stores the `userId`, the `problemId`, the `currentStep` they are on, and the actual text content for all 5 steps. This is what allows the 'Resume' functionality."
-

3. Server Layer (The Logic)

"On the backend, I used Express and organized it into Controllers and Middlewares:

- **Authentication:** Every request is protected by an `authMiddleware` that checks the JWT token in cookies.
 - **Fetching Problems:** When the user sees the list of problems, the server doesn't just send the titles. It compares the problems with the user's `Drafts` and `Submissions` collections to tag them as '**NEW**', '**IN_PROGRESS**', or '**SOLVED**'.
 - **Draft Logic:** I created a `saveDraft` controller. It uses a MongoDB `upsert` (Update or Insert). If a draft doesn't exist for that user and problem, it creates one. If it does, it updates the specific step the user just finished."
-

4. Client Layer (The User Experience)

"On the frontend, I used **React** with **Redux Toolkit Query (RTK Query)**:

- **Problems Page:** It calls `getAllProblems`. Depending on the status tag from the server, the UI highlights the card differently (e.g., green for solved, orange for in-progress).
 - **Problem Page:** When a user clicks a problem, we fetch the problem details **and** any existing draft. If there's a draft, the UI automatically fills in the text areas for the steps they already completed.
 - **The Workspace:** This is a dynamic component. When the user clicks '**Save & Next**', it triggers a mutation that updates the DB. I also implemented a 2-minute timer for hints to encourage users to think before they peek!"
-

5. The "Save Draft" Flow (Step-by-Step Example)

"If you ask me how a single 'save' happens:

1. The user types their 'Understanding' in the `Workspace` component.
 2. When they click 'Save & Next', the frontend sends the `problemId`, the `step name`, and the `content` to the `/api/drafts/save` endpoint.
 3. The backend validates the content (like checking if it's long enough).
 4. It saves it to the `Drafts` collection using the user's ID from the JWT.
 5. The frontend then updates the `activeStep` state to move them to the 'Breakdown' step.
 6. This ensures that even if they refresh the page or lose internet, their hard work is safe in our DB."
-

6. Technical Highlight (What I'm proud of)

"I'm particularly proud of how I handled the 'Status' tagging. By joining the data on the server-side, the frontend stays 'dumb' and fast—it just receives a list of problems with a status string and renders it. It keeps the UI very responsive."